

`user["firstName"]` ~ accessing Object Elements.

functions ↴ remember to use these quotation while accessing something.

```
function sum(a, b) {  
    const sumValue = a + b;  
    return sumValue;  
}
```

```
const value = sum(1, 2);  
console.log(value);
```

JavaScript Callbacks → Function as an Argument

```
function sum(num1, num2, fnToCall) {  
    let result = num1 + num2;  
    fnToCall(result);  
}
```

```
function displayResult(data) {  
    console.log("Result of the sum is: " + data);  
}
```

```
function displayResultPassive(data) {  
    console.log("Sum's result is: " + data);  
}
```

// you are only allowed to call one function after this
// how will you displayResult of a sum.

```
const ans = sum(1, 2, displayResult);
```

Another Type.

```
function calculateArithmetic(a, b, arithmeticFinalFunction) {
```

```
    const ans = arithmeticFinalFunction(a, b);
```

```
    console.log(ans);
```

```
}
```

```
function sum(a, b) {
```

```
    return a + b;
```

```
}
```

```
const value = calculateArithmetic(1, 2, sum);
```

```
console.log(value);
```

↳ will be discussed later in detail.

different / Various ways of creating function in javascript.

1) function greet(name) {

```
    return "Hello, " + name;
```

```
}
```

↳ hoisted

↳ is a JS default behavior of moving

declaration to the top of their scope (before code execution). Remember
declaration are hoisted not initialization.

2) Function Expression (Anonymous or Named)

```
const greet = function(name) {  
    return "Hi , " + name;  
};
```

↳ not hoisted & since const is there so ; is preferred.

3) Arrow Function (ES6+)

```
const greet = (name) => {  
    return "Hey , " + name;  
};
```

// or if single return

```
const greet = name => "Yo , " + name;
```

4. Immediately Involved Function Expression (IIFE)

```
(function(name) {  
    console.log("Welcome , " + name);  
})("Eve"); → // Welcome , Eve
```

Common JavaScript Web APIs

```
setTimeout(() => {  
    console.log("Runs after 2 seconds");  
}, 2000);
```

```
const intervalId = setInterval(() => {  
    console.log("Runs every 3 seconds");  
}, 3000);
```

// To stop the interval after 10 seconds.

```
setTimeout(() => clearInterval(intervalId), 10000);
```

// Networking API

```
fetch("https://jsonplaceholder.typicode.com/posts/1")
```

- then(response => response.json())

- then(data => console.log("fetched data:", data));

JavaScript Callback ~>